

- JS1-声明
 - 1 var
 - 1.1 作用域
 - 1.2 提升
 - 1.2.1 变量提升
 - 1.2.2 函数提升
 - 1.2.3 优先级差异
 - 2 let
 - 2.1 作用域
 - 2.2 暂时性死区 (Temporal Dead Zone - TDZ)
 - 3 const
 - 3.1 常量不可变性
 - 4 对比总结

JS1-声明

1 var

var 是ES6之前JavaScript唯一的变量声明方式。

1.1 作用域

var声明的变量在其所在的函数内有效。如果在函数外声明，则为全局变量。这意味着**var**声明的变量在 **if**、**for**、**while**等块级语句中声明时，其作用域不会被限制在这些块中，而是会“泄露”到其所在的函数作用域或全局作用域。

```
// if的泄露问题
if(true){
    var x = 10;
}
console.log(x); //输出10

// for,while循环的泄露问题
for (var i=0;i<3;i++){
    //.....
}
console.log(i); //输出3
```

考点：

1. 为什么 `var` 声明的变量没有块级作用域？

这是 `var` 的设计缺陷，它不遵循块级作用域的规则，导致变量容易在不期望的地方被访问或修改。

2. 经典的循环闭包问题

以下JavaScript代码执行的输出结果是：

```
for (var i = 0; i < 10; i++) {  
    setTimeout(function () {  
        console.log(i);  
    }, 1000);  
}  
// 输出10个10
```

原因：`setTimeout` 是异步执行的，当回调函数执行时，`for` 循环已经结束，`i` 的值已经变为10。由于 `var` 没有块级作用域，所有回调函数共享同一个 `i` 变量。

1.2 提升

1.2.1 变量提升

`var` 声明的变量会发生变量提升，即在代码执行之前，变量的声明会被“提升”到其所在作用域的顶部。但需要注意的是，**只有声明会被提升，赋值操作不会被提升。**

```
// var 变量提升  
console.log(a); // undefined  
var a = 5;  
console.log(a); // 5
```

上述代码等价于

```
var a; // 提升声明并初始化为undefined  
console.log(a); // undefined  
a = 5;  
console.log(a); // 5
```

考点：

1. 为什么 `console.log(a)` 会输出 `undefined` 而不是报错？

因为 `var a` 的声明被提升了，所以在 `console.log` 执行时，`a` 变量已经存在，只是还没有被赋值，所以是 `undefined`。

2. 变量提升的“坏处”是什么？

代码可读性差：变量可以在声明之前被使用，导致代码的执行结果与阅读顺序不一致，容易产生歧义和难以追踪的bug。

意外的全局变量：如果在函数内部不使用 `var` 声明变量，该变量会自动成为全局变量，污染全局作用域。

1.2.2 函数提升

与变量类似，**函数声明** 也会被提升到其作用域的顶部。不同的是，函数声明不仅会提升其名称，还会提升整个函数体。因此，即使在函数声明之前调用该函数，代码仍然可以正常执行。

```
console.log(foo()); // 输出: Hello!
function foo() {
  return "Hello!";
}
```

上述代码等价于

```
// 函数提升
function foo() {
  return "Hello!";
}
console.log(foo()); // 输出: Hello!
```

与函数声明不同，**函数表达式** 不会被提升。这是因为函数表达式中的函数定义是赋值操作，而赋值操作不会被提升。

```
console.log(baz); // 输出: undefined
var baz = function() {
  return "Hi!";
}
```

```
};  
console.log(baz()); // 输出: Hi!
```

上述代码等价于

```
var baz;  
console.log(baz); // 输出: undefined  
baz = function() {  
    return "Hi!";  
};  
console.log(baz()); // 输出: Hi!
```

在这段代码中，`baz`是一个函数表达式，它的声明被提升了，但函数赋值操作没有被提升，因此在第一次 `console.log` 时，`baz`的值是 `undefined`。只有在函数赋值完成后，`baz`才能被调用。

1.2.3 优先级差异

函数提升优先于变量提升，且函数声明会覆盖同名变量声明（但不会覆盖变量赋值）。

```
console.log(bar); // f bar() {}（函数提升优先，变量声明被忽略）  
var bar = 10;  
console.log(bar); // 10（变量赋值覆盖函数）  
function bar() {} // 函数声明
```

上述代码等价于

```
// 1. 函数提升  
function bar() {}  
  
// 2. 变量声明（被函数声明覆盖，无效）  
// var bar;（忽略，因同名函数已存在）  
  
// 3. 执行代码  
console.log(bar); // f bar() {}  
bar = 10; // 变量赋值，覆盖函数  
console.log(bar); // 10
```

综合考察：变量与函数的优先级

```
var a = 10;  
function a() {}
```

```
console.log(typeof a); // 输出什么?
```

上述代码等价于

```
function a() {} // 函数提升
// var a;          变量 var a 的声明会因重复而被忽略。
a = 10; // 变量赋值，覆盖函数
console.log(typeof a); // 输出 'number'
```

2 let

let是ES6引入的新的变量声明方式，旨在解决 **var**的诸多问题，特别是引入了“块级作用域”和“暂时性死区”。

2.1 作用域

let声明的变量只在声明它的代码块（**{}**）内有效。这使得变量的作用域更加清晰，避免了 **var**带来的变量污染问题。

```
if(true){
    let x = 10;
}
console.log(x); //ReferenceError: x is not defined

for (let i=0;i<3;i++){
    // .....
}
console.log(i); //ReferenceError: i is not defined
```

考点：

如何解决 **var**的循环闭包问题？

使用`let`声明循环变量即可。因为`let`为每次循环都创建了一个独立的块级作用域，确保每个回调函数都捕获到当前循环迭代的`i`值。

```
for (let i = 0; i < 5; i++) {
    setTimeout(function() {
```

```
    console.log(i);
  }, 1000);
}
// 输出: 0, 1, 2, 3, 4
```

2.2 暂时性死区 (Temporal Dead Zone - TDZ)

`let`和 `const`声明的变量也存在变量提升，但与 `var`不同的是，它们在声明之前是不可访问的。从代码块的开始到变量声明语句之间的区域，被称为“暂时性死区”。在此区域内访问变量会抛出 `ReferenceError`。

```
console.log(a); // ReferenceError: Cannot access 'a' before initialization
let a = 1; // 词法作用域
```

JavaScript引擎在解析代码时，会先扫描整个作用域，找到所有的 `let`和 `const`声明，并将其放入一个“未初始化”的状态。当代码执行到变量声明的那一行时，变量才会被初始化并赋值。在“未初始化”状态到初始化完成之前的这段时间，就是暂时性死区。

3 const

`const`也是ES6引入的，用于声明常量。它与 `let`有很多相似之处，例如都具有块级作用域和暂时性死区，但其核心特性是“常量不可变”。

3.1 常量不可变性

`const`声明的变量在声明时必须进行初始化，并且一旦赋值后，其值就不能再被重新赋值。

```
const PI = 3.14;
PI = 3.14159; // TypeError: Assignment to constant variable.

const obj = { a: 1 };
obj = {}; // TypeError: Assignment to constant variable.
```

考点：

`const`声明的对象或数组可以修改吗？**可以**。`const`保证的是变量指向的内存地址不变，而不是内存地址中存储的值不变。对于引用类型（对象、数组），`const`只保证变量名指向的那个引用地址不变，但该引用地址指向的对象或数组内部的属性或元素是可以修改的。

```
const arr = [1, 2, 3];
arr.push(4); // 允许，arr 仍然指向同一个数组
console.log(arr); // [1, 2, 3, 4]

const user = { name: "Alice" };
user.name = "Bob"; // 允许，user 仍然指向同一个对象
console.log(user); // { name: "Bob" }
```

4 对比总结

特性	var	let	const
作用域	函数作用域/全局作用域	块级作用域	块级作用域
变量提升	有	有（声明提升，但存在暂时性死区）	有（声明提升，但存在暂时性死区）
重复声明	允许	不允许	不允许
修改	允许	允许	不允许（对引用类型可修改内部）
初始值	可不初始化	可不初始化	必须初始化